

**UNIVERSITÉ GRENOBLE ALPES
INFORMATIQUE, MATHÉMATIQUES ET
MATHÉMATIQUES APPLIQUÉES
MASTER INFORMATIQUE — PARCOURS
INFORMATIQUE & COGNITION (IC)**



Rapport de Projet - LifeStories

*Implémentation du composant questionnaire d'un outil de
collecte de données biographiques rétrospective*

Étudiant: *Tran Phuc Tin Truong*

Adrien Van-Robays

Encadrant: *Marlène Villanova*

Benjamin Fontaine

Projet tuteuré

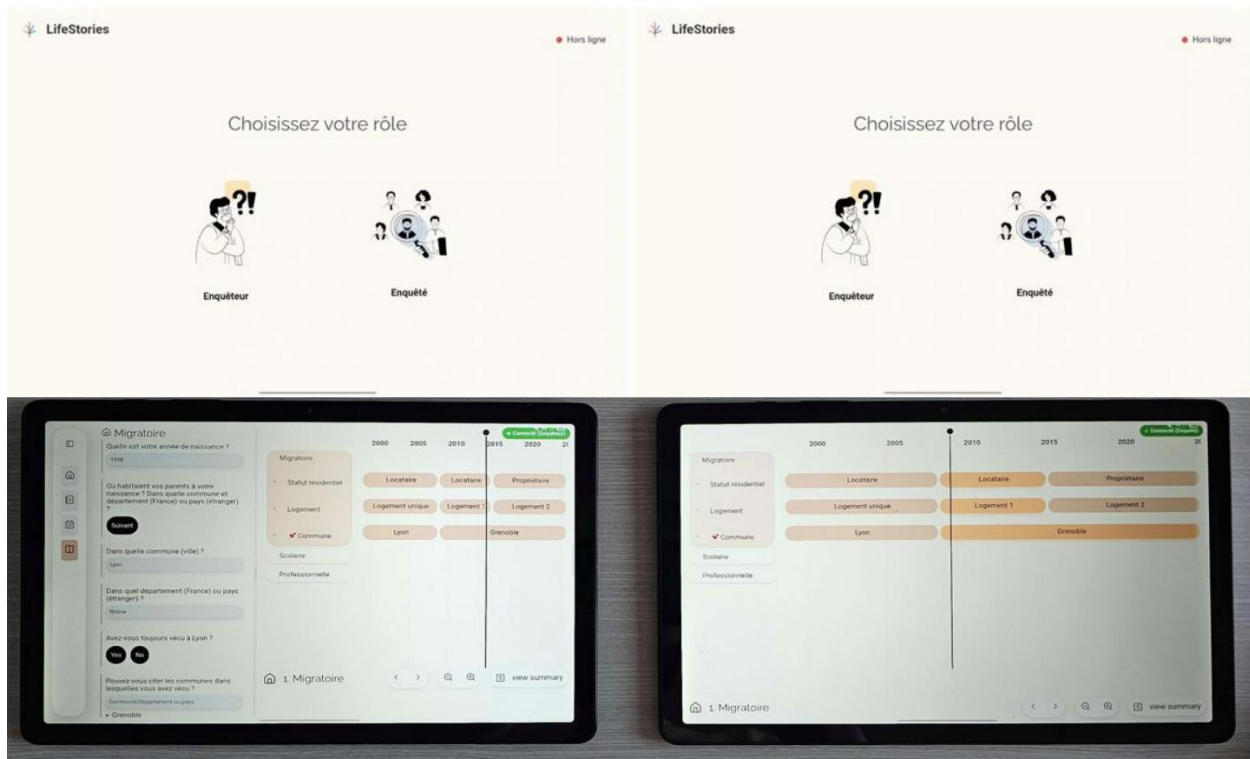
SOMMAIRE

1. Introduction et Contexte du Projet:	3
1.1. Contexte et objectif du projet	3
1.2. Périmètre de notre travail et objectifs	4
2. Organisation du Travail, Répartition et Méthodologie	4
2.1. Répartition des responsabilités	4
2.2. Outils de collaboration et Suivi	5
2.3. Analyse Critique de notre Organisation	5
3. Architecture Technique et Choix des Outils	5
3.1. Stack Technologique sélectionnée	6
4. Focus Technique : Le Parser et le Traitement Algorithmique du Prototype	7
4.1. Le Pont Sémantique (semantic_model.json)	7
4.2. Le Mécanisme de Fallback (Le défi des dates)	8
4.3. Génération des blocs temporels	8
4.4. Orchestration et rôles des composants	9
5. Analyse Critique de notre Ingénierie (Défis et Axes d'améliorations)	10
5.1. TDD et Tests Unitaires	10
5.2. Architecture et Dette Technique (main.js)	11
6. Conclusion et Perspectives	11

1. Introduction et Contexte du Projet:

1.1. Contexte et objectif du projet

La recherche en sciences sociales, particulièrement la sociologie et la démographie, s'appuie fréquemment sur des enquêtes biographiques afin de retracer avec précision les parcours de vie des individus. Le projet global dans lequel s'inscrit notre travail vise à moderniser cette collecte sur le terrain en remplaçant les fiches papier traditionnelles par un dispositif numérique, déployé nativement sur **tablettes tactiles**. L'architecture matérielle cible prévoit ainsi deux tablettes fonctionnant en parallèle : l'une dédiée à l'enquêteur pour la conduite de l'entretien, et l'autre face à l'enquêté, lui permettant de voir sa trajectoire de vie se construire graphiquement et en temps réel, favorisant ainsi la remémoration de son passé. Le prototype « LifeStories », point de départ de notre travail est illustré dans la figure suivante (à gauche la vue de l'enquêteur et à droite celle de l'enquêté)



(Schéma conceptuel du dispositif double-tablette de l'écosystème LifeStories)

1.2. Périmètre de notre travail et objectifs

Bien que l'infrastructure finale cible les tablettes Android, le cœur de notre mission s'est concentré de manière stricte sur la chaîne de traitement logicielle permettant d'alimenter ce dispositif. En effet, l'application existante a été conceptualisée dans le but d'explorer la faisabilité du hors-ligne entre les deux tablettes et d'explorer les interactions existantes entre le composant timeline et le composant questionnaire. Cependant le questionnaire reste actuellement « codé en dur ». Nos contributions majeures s'articulent donc autour de trois jalons opérationnels :

1. **La modélisation du questionnaire** : La conception et la structuration d'une enquête biographique test via l'outil **KoboToolbox** à partir d'un questionnaire rédigé à la main (document Word).
2. **Le développement de l'Analyseur Syntaxique (Parser)** : La création d'un moteur capable de lire, d'interpréter et de traduire de manière générique les fichiers de données bruts issus du questionnaire.
3. **La consolidation du Prototype Final** : L'implémentation d'une interface applicative unifiée, connectant le questionnaire issu de KoboToolbox à un module de visualisation chronologique (**Timeline**), constituant la preuve de concept fonctionnelle de la solution.

2. Organisation du Travail, Répartition et Méthodologie

Pour mener à bien ce projet dans les délais impartis, nous avons dû mettre en place une organisation stricte entre les deux membres de l'équipe.

2.1. Répartition des responsabilités

Afin d'éviter de nous marcher sur les pieds et de paralléliser le développement, nous avons scindé le projet en deux pôles d'expertise :

- **Le Pôle Formulaire et Modèle de Données (KoboToolbox & Enketo)** : Responsable de la traduction du questionnaire biographique en standard XForms via KoboToolbox, de l'exportation du modèle de données XML natif, et de l'intégration du moteur de rendu Enketo Core pour générer le HTML interactif.
- **Le Pôle "Parser", Traitement et Visualisation (JavaScript & vis-timeline)** : Responsable de l'écoute des événements du DOM, de la transformation (parsing) du fichier

XML brut en objets manipulables, de la logique de calcul des dates, et de l'affichage dynamique sur la Timeline.

2.2. Outils de collaboration et Suivi

Nous avons utilisé un dépôt Git (hébergé sur GitLab/GitHub) pour versionner notre code. Cela nous a permis d'isoler les expérimentations (branches) sans casser le prototype principal.

- *Semaines 1-2* : Définition des besoins et maquettage.
- *Semaines 3-5* : Implémentation du formulaire Kobo et intégration d'Enketo.
- *Semaines 6-8* : Développement du Parser XML et intégration de vis-timeline.
- *Semaines 9-10* : Synchronisation temps réel, débogage, et rédaction de la documentation.

2.3. Analyse Critique de notre Organisation

Bien que la séparation des tâches ait été bénéfique initialement, notre gestion de projet a souffert d'un défaut de communication technique, ce qui a rendu la phase d'intégration beaucoup plus longue que prévu.

- **Analyse** : Nous avons commis l'erreur de travailler en "silos" trop étanches. Le Pôle Visualisation développait ses scripts en s'attendant à une certaine structure XML, tandis que le Pôle Formulaire générait un XML avec des identifiants (ex : Q_commune_n) différents ou non standardisés.
- **Solution corrective** : Nous aurions dû rédiger un "Contrat d'Interface" (API Specification) strict dès la semaine 2, listant précisément les balises attendues dans le fichier output_model.xml. Cela aurait évité de réécrire une grande partie du Parser en semaine 8. C'est d'ailleurs ce que nous avons fini par mettre en place pour débloquer la situation, en définissant un vocabulaire commun basé sur des préfixes stricts (ex : A_ pour les attributs, T_ pour les trajectoires, E_ pour les événements, SD_ pour *Start Date*, ED_ pour *End Date*, Q_ pour les questions existantes, etc.). L'implémentation de cette nomenclature sera détaillée plus en profondeur dans la section 4.1.

3. Architecture Technique et Choix des Outils

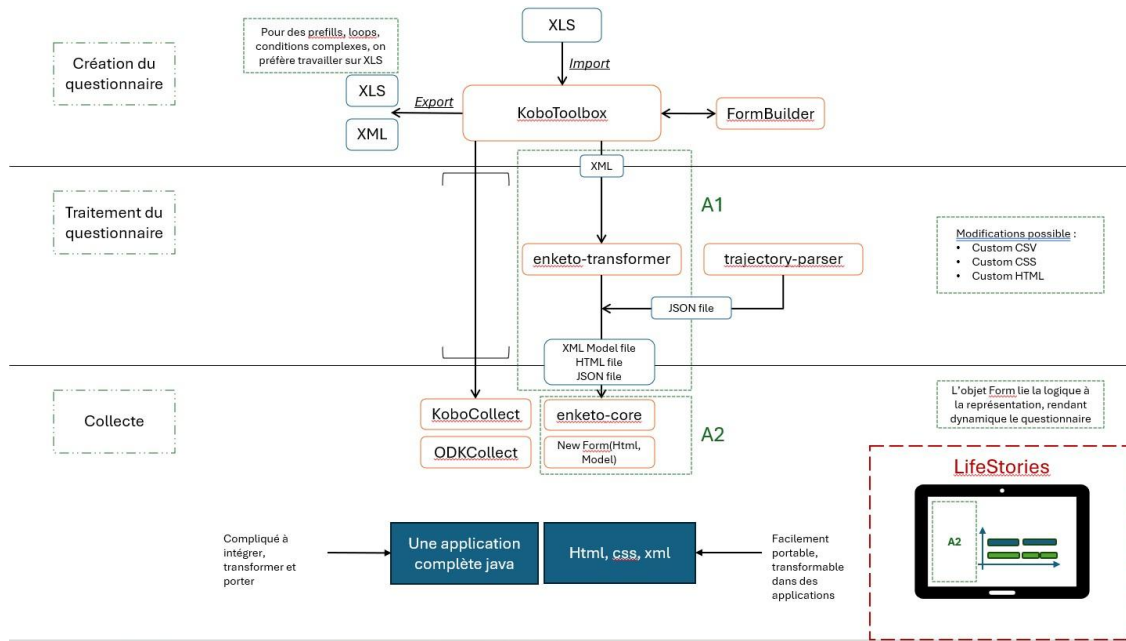
La contrainte d'avoir un outil fonctionnant en local, hors-ligne et hautement réactif a dicté nos choix architecturaux vers une structure de Single Page Application (SPA). Ces choix techniques ont également été motivés par le prototype existant « LifeStories ». En effet, plutôt que de réécrire l'intégralité de l'application, notre démarche a consisté à nous adapter à cet écosystème préexistant afin d'isoler et d'interconnecter deux composants principaux : l'instance du questionnaire et le module de la timeline (flux de visualisation chronologique).

3.1. Stack Technologique sélectionnée

- **Le Bundler (Vite) :** Nous avons choisi Vite plutôt que Webpack ou Create React App. Le projet ne nécessitait pas la lourdeur d'un framework massif. Vite nous a permis de conserver du Vanilla JavaScript (plus léger, plus proche du DOM) tout en bénéficiant d'un serveur de développement ultra-rapide avec Hot Module Replacement (HMR).
- **Le Moteur de rendu (Enketo Core) :** Plutôt que de recoder un formulaire complexe en HTML, nous nous sommes appuyés sur Enketo Core. Cet outil puissant prend un fichier XML généré par KoboToolbox et le transforme en un formulaire web interactif, gérant nativement les sauts logiques et les groupes répétés (*repeat groups*).
- **La Visualisation (vis-timeline) :** Pour le calendrier, notre choix s'est porté sur vis-timeline. Ses avantages étaient clairs : rendu basé sur le DOM, gestion native des regroupements (*Groups*), du chevauchement (*Stacking*), et des barres de temps personnalisées (*Custom Time Bar*) indispensables pour notre fonction de Synthèse.

Légende et commentaire de la figure : Ce schéma illustre l'architecture logicielle globale et le flux de données entre les différents modules. Il met en évidence la chaîne de traitement à travers deux travaux majeurs réalisés :

1. **Le Générateur (A1) :** Cet outil prend en charge le questionnaire conçu sur KoboToolbox et prépare son affichage en automatisant la génération des fichiers HTML et XML nécessaires.
2. **L'Instance du Client (A2) :** Il s'agit de l'instance du questionnaire qui s'exécute sous Enketo Core. La logique conditionnelle, le dynamisme et les interactions de saisie sont entièrement gérés par cette librairie, avant d'envoyer les données vers le module de visualisation.



(Schéma de l'Architecture Logicielle globale et flux de données entre KoboToolbox, Enketo et le module de Visualisation)

4. Focus Technique : Le Parser et le Traitement Algorithmique du Prototype

C'est dans l'interaction en temps réel entre le formulaire KoboToolbox et la timeline que réside la plus grande complexité technique du projet, gérée au sein du fichier main.js. À chaque modification effectuée par l'utilisateur (événement input ou change), l'arbre XML complet est réévalué par notre analyseur.

4.1. Le Pont Sémantique (semantic_model.json)

Pour éviter de « hardcoder » (figer) les noms des variables Kobo dans le script de visualisation, nous avons introduit un modèle sémantique externe sous forme de fichier JSON. Ce fichier sert de lien fondamental entre le vocabulaire standardisé que nous avons spécifiquement ajouté lors de la conception du questionnaire (comme évoqué dans la section 2.3) et le composant graphique de la Timeline.

Concrètement, il agit comme un dictionnaire de traduction : il établit une correspondance dynamique entre la nomenclature préfixée du XML (par exemple A_ pour un attribut, T_ pour une trajectoire, SD_ pour *Start Date*, ED_ pour *End Date*) et la structure de données attendue par la vue. Par exemple, ce modèle stipule clairement au script que l'attribut visuel "Commune" doit aller

puiser son contenu textuel dans le nœud XML `V_commune_n`, et extraire sa date de début via la balise associée `SD_cn`.

```
{
  "id": "E_commune_n",
  "question": "Q_commune_n",
  "value": "V_commune_n",
  "descriptors": [
    "D_pays_cn",
    "D_dept_cn"
  ],
  "startDate": "SD_cn",
  "endDate": null,
  "status": null
}
```

(Capture d'écran d'un extrait du fichier `semantic_model.json`)

Légende de l'image : Extrait du modèle sémantique illustrant le mappage entre les identifiants standardisés du questionnaire KoboToolbox et les propriétés attendues par la Timeline.

4.2. Le Mécanisme de Fallback (Le défi des dates)

- **Le problème :** Lors d'un entretien biographique, on se souvient rarement d'une date exacte. Les répondants entrent fréquemment un âge ou une approximation ("J'ai déménagé à 15 ans"). De ce fait, les champs de dates canoniques (`SD_cn`) sont très souvent vides dans le XML, empêchant la timeline de positionner les repères temporels.
- **La solution (Algorithme de Fallback) :** Lorsque le Parser rencontre une date vide, l'algorithme remonte à l'élément parent (`parentNode`), scanne tous les nœuds frères (`childNodes`), et recherche via des expressions régulières des mots-clés spécifiques (ex : `age_arrivee`, `annee_debut`). S'il trouve un âge (ex : 15), il calcule dynamiquement la date d'insertion : $StartDate = Année\ de\ naissance\ (extraite\ au\ préalable) + Âge$.

4.3. Génération des blocs temporels

Pour qu'une chronologie biographique soit lisible, la représentation sous forme de simples points temporels est souvent insuffisante ; il est nécessaire d'afficher des durées continues (*Ranges*). Cet algorithme de post-processing vient donc en complément d'un traitement de date plus direct. En effet, lorsque certaines questions du formulaire fournissent explicitement à la fois une date de début et une date de fin, la création du bloc temporel est immédiate. En revanche, lorsque les

données ne fournissent que des points de départ successifs, notre algorithme prend le relais : il isole les événements par catégorie (trajectoire), les trie chronologiquement, et chaîne automatiquement la date de début de l'événement $n+1$ pour en faire la date de fin de l'événement n . Ce chaînage garantit un rendu graphique parfaitement contigu sur la Timeline, évitant ainsi les trous temporels incohérents ou inexpliqués dans l'histoire de l'enquête.

4.4. Orchestration et rôles des composants

Afin de clarifier la dynamique de ce processus et de définir les responsabilités de chaque composant du système (de la saisie des données jusqu'à leur restitution visuelle), nous avons structuré notre architecture logicielle autour de plusieurs éléments clés interagissant en temps réel. Dans cette orchestration, chaque fichier joue un rôle architectural précis :

- **output_form.html (La Vue)** : Propulsé par Enketo, c'est l'interface du formulaire. Il s'agit de la couche visuelle et interactive que l'utilisateur voit, manipule et qui capture les saisies en direct.
- **output_model.xml (La Base de données locale)** : Générée par KoboToolbox, elle agit sous le capot pour stocker l'ensemble des réponses saisies de manière hiérarchique et structurée.
- **semantic_model.json (La Boussole)** : C'est le dictionnaire de traduction. Il dicte dynamiquement au script quelles variables XML il doit cibler et extraire pour pouvoir peindre la Timeline correctement.
- **main.js (Le Cerveau / Contrôleur)** : C'est l'orchestrateur du système. Il écoute en permanence les événements de La Vue, interroge La Base de données locale en s'appuyant sur La Boussole, calcule algorithmiquement les dates manquantes et synchronise l'affichage final de la Timeline.

Questionnaire (Enketo)

JSON

I. Début de l'enquête
 Bienvenue dans cette enquête biographique LifeStory.
 * En quelle année êtes-vous né(e) ?
 2002

II. Trajectoire résidentielle

» A_Commune

» Introduction Commune
 Pour commencer, nous allons décrire ensemble les différentes communes pendant lesquelles vous avez vécu au moins un an depuis votre naissance.
 (Il est parfois possible d'afficher de façon habituelle dans deux lieux de résidences en même temps (commune, logement). N'oubliez pas de les mentionner dans la liste des communes où vous avez habité.)

» E_commune_1
 Où habitaient vos parents à votre naissance ?
 * Pays
 France
 * Commune :
 Grenoble

Calendrier de vie

	2005	2010	2015	2020	2025
Commune	Grenoble		Paris		
Logement	maison				
Status Res	bocataire				
Niveau Scolaire					
Type Etablissement					
Activité					
Status Emploi					
Type Emploi					
Activité Entreprise					
Grp Traj Résidentielle					
Grp Traj Scolaire					
Grp Traj Pro					

Synthèse

GRP TRAJ RESIDENTIELLE -> COMMUNE
Grenoble
 De : 2002 à 2020

GRP TRAJ RESIDENTIELLE -> LOGEMENT
maison
 De : 2002 à 2020

(Capture d'écran du Prototype fonctionnel montrant le formulaire Enketo à gauche, la Timeline dynamique et le panneau de Synthèse à droite)

Légende de la figure : Cette capture d'écran illustre l'interface utilisateur unifiée de notre preuve de concept en action. À gauche, le panneau intègre le formulaire dynamique Enketo Core dédié à la saisie des données biographiques par l'enquêteur. À droite, le système affiche simultanément la Timeline interactive synchronisée en temps réel et le panneau de Synthèse transversale, offrant ainsi une double rétroaction visuelle immédiate et fluide.

5. Analyse Critique de notre Ingénierie (Défis et Axes d'améliorations)

Le but de ce projet n'était pas seulement de livrer un produit fonctionnel, mais de comprendre comment concevoir un logiciel maintenable. Plusieurs décisions architecturales méritent d'être sévèrement critiquées.

5.1. TDD et Tests Unitaires

- **Le Constat :** Nos algorithmes de calcul de dates se sont complexifiés avec le temps. Un bug critique a illustré le danger d'une validation purement manuelle : l'omission d'un simple transtypage (*String* vers *Integer*) lors de l'extraction de l'âge depuis le XML a faussé les calculs. Au lieu d'une date valide, le système a généré des valeurs NaN ou 1970, détruisant l'affichage de la Timeline.

- **L'Analyse Critique** : Résoudre cette erreur basique a exigé de nombreuses heures de débogage manuel fastidieux (console.log). L'adoption d'une démarche de développement piloté par les tests (TDD avec **Jest** ou **Vitest**) dès le début du projet aurait permis de détecter ce comportement instantanément face à des données factices. *Leçon retenue* : le temps apparemment gagné en évitant l'écriture des tests initiaux est largement perdu lors de la phase de débogage final.

5.2. Architecture et Dette Technique (main.js)

- **Le Constat** : Actuellement, toute la logique de l'application (parsing XML, calculs des dates, manipulation de l'interface et configuration de la timeline) est concentrée dans un seul fichier massif (main.js).
- **L'Analyse Critique** : Cette centralisation crée un code "spaghetti" fragile, violant le principe de responsabilité unique. Pour garantir la maintenabilité et la transférabilité du projet, une refonte vers l'architecture **MVC (Modèle-Vue-Contrôleur)** est indispensable. Il faut impérativement séparer le traitement des données (XmlParser.js) de la gestion de l'affichage graphique (TimelineRenderer.js). L'analyse critique peut être tempérée par le temps nécessaire et l'objectif qui est plus une preuve de concept.

6. Conclusion et Perspectives

Le prototype que nous livrons aujourd'hui répond bien aux attentes de départ. Il offre aux chercheurs une interface à deux panneaux très pratique : la saisie des données via KoboToolbox et Enketo alimente et met à jour directement la chronologie visuelle. Pour la suite, une bonne idée d'amélioration serait de reprendre le design du tout premier LifeStories. Par exemple, afficher les questions qui se répètent (repeat groups) sous forme d'onglets rendrait la navigation beaucoup plus agréable pour l'utilisateur.

Humainement et techniquement, ce projet nous a plongés dans le vrai quotidien des développeurs. On retiendra surtout une décision importante que nous avons prise : celle de ne pas modifier le gros code existant de LifeStories, mais de construire notre outil "à côté", de manière isolée. Ce stage nous a aussi appris à gérer les données en temps réel et à comprendre pourquoi il ne faut pas prendre de raccourcis techniques quand on est dans l'urgence.

Pour finir, nos points faibles — comme le fait que notre code soit encore un peu trop centralisé, ou le fait d'avoir écrit les tests unitaires trop tard — ne sont pas des échecs. Ce sont de super leçons pour l'avenir. Ces erreurs de parcours nous aideront à devenir des informaticiens mieux organisés et plus prudents. Aujourd'hui, nous laissons derrière nous des bases techniques solides, prêtes à être reprises et améliorées par les prochains étudiants du LIG !